



데이터 전처리

⚙ Progress	완료
☰ Type	

1. 데이터 디렉토리

- Usage: 공유 구글 드라이브에 원본 데이터를 저장하며, 학습 시에는 로컬에 가져와서 사용한다.
- 아래의 디렉토리 구조에 따라 데이터를 저장한다.

▼ 디렉토리 구조

```
- data
  - raw
    - abnormal
    - normal
  - augmented
    - abnormal
  - labeled
    - abnormal
    - normal
  - splits
    - split_manifest.json      # train/valid/test 분할
정보
  - samples
    - conversation
    - prefix
  - final                      # 실제 학습에 쓰는 최종 JSO
NL
  - train.jsonl
  - valid.jsonl
  - test.jsonl
```

2. 파일명 규칙 및 내부 스키마

2-1. 원본 통화 json 파일

- 아래의 형식에 맞춰 파일명을 할당한다.

⇒ `raw_<conversation_id>.json`

- 6자리로 0-padding
 - normal: 100000 번대 사용
 - abnormal: 200000 번대 사용
- 원본 통화 json 파일의 스키마는 아래와 같다.

▼ json 스키마

```
{
  "conversation_id": <conversation_id>,
  "conversation_label": <conversation_label>,
  "utterances": [
    {
      "idx": 0,
      "speaker": "S1",
      "text": "....."
    }
    {
      ...
    }
  ]
}
```

2-2. 증강 통화 json 파일

- 아래의 형식에 맞춰 파일명을 할당한다.

⇒ `aug_<conversation_id>.json`

- 증강은 추가적으로 7번째 자리에서 표기한다.
 - ex) 원본이 177이면 증강본은 2001771, 2001772, 2001773
- 증강 통화 json 파일의 스키마는 원본 통화 json 파일의 스키마와 동일하다.

2-3. 문장별 라벨링 완료본

- 아래의 형식에 맞춰 파일명을 할당한다.

⇒ `labeled_<conversation_id>.json`

- 문장별 라벨링이 된 json 파일의 스키마는 아래와 같다.

▼ json 스키마

```
{
  "conversation_id": <conversation_id>,
  "conversation_label": <conversation_label>,
  "utterances": [
    {
      "idx": 0,
      "label": 0,
      "speaker": "S1",
      "text": "....."
    }
    {
      ...
    }
  ]
}
```

2-4. split 정보 파일

- 아래의 형식에 맞춰 파일명을 할당한다.

⇒ `split_manifest_v1.json`

- split 정보 json 파일의 스키마는 아래와 같다.

▼ json 스키마

```
{
  "train": [200177, 200188, 100431, 100432],
  "valid": [200501, 100601],
  "test": [200701, 100801]
}
```

2-5. 샘플 jsonl 파일

- jsonl 파일에는 한 줄에 json 객체 1개를 담고 있으며, 학습 샘플들을 하나의 파일에 모은 것이다.
- 최종적으로 학습에 사용할 파일은 samples 디렉토리에 있는 파일들을 가공하여 생성 후 final 디렉토리에 저장된다.

2-5-1. samples/conversation

- 아래의 3개의 파일이 존재한다.

⇒ conversation_train_v1.jsonl

⇒ conversation_valid_v1.jsonl

⇒ conversation_test_v1.jsonl

- 하나의 파일에 train, valid, test에 쓰일 json 파일이 모두 들어 있다.

- conversation jsonl 파일은 아래와 같다.

▼ jsonl 내용

```
{"sample_id": "conversation_200001", ...}  
{"sample_id": "conversation_200002", ...}  
{"sample_id": "conversation_100001", ...}
```

2-5-2. samples/prefix

- 아래의 3개의 파일이 존재한다.

⇒ prefix_train_v1.jsonl

⇒ prefix_valid_v1.jsonl

⇒ prefix_test_v1.jsonl

- 하나의 파일에 train, valid, test에 쓰일 json 파일이 모두 들어 있으며, 하나의 파일 내부에 하나의 json에서 생성된 prefix 단위 파일들이 모두 존재한다.

- prefix jsonl 파일은 아래와 같다.

▼ jsonl 내용

```
{"sample_id": "prefix_200001_0004", ...}  
{"sample_id": "prefix_200001_0009", ...}
```

```

{"sample_id": "prefix_200001_0012", ...}
{"sample_id": "prefix_100001_0004", ...}
{"sample_id": "prefix_100001_0009", ...}
{"sample_id": "prefix_100001_0014", ...}
{"sample_id": "prefix_100001_0017", ...}

```

2-6. final 파일

- 실제로 학습에 넣는 최종 파일로, conversation 샘플과 prefix 샘플을 합쳐서 넣는다.
- 아래의 3개 파일이 존재한다.

⇒ `train.jsonl`

⇒ `valid.jsonl`

⇒ `test.jsonl`

- final jsonl 파일은 아래와 같다.

▼ jsonl 내용

```

{"sample_id": "conversation_200001", "sample_type": "con
versation", ...}
{"sample_id": "conversation_100001", "sample_type": "con
versation", ...}
{"sample_id": "prefix_200001_0004", "sample_type": "pref
ix", ...}
{"sample_id": "prefix_200001_0009", "sample_type": "pref
ix", ...}
{"sample_id": "prefix_100001_0004", "sample_type": "pref
ix", ...}

```

3. 비정상 데이터 증강

- 하나의 비정상 데이터에 대하여 아래의 프롬프트와 Gemini 2.5 Flash 모델을 활용하여 3개의 비정상 데이터를 증강한다.

▼ 프롬프트

Role

당신은 한국어 보이스피싱 통화 데이터 증강 전문가입니다.

주어진 원본 통화 데이터를 바탕으로, 의미와 사기 전략은 유지하되 표현만 자연스럽게 바꾼 새로운 대화 데이터를 생성해야 합니다.

Objective

원본 대화와 동일한 보이스피싱 의도, 핵심 흐름, 설득 전략을 유지하면서,
어휘, 문장 구조, 말투, 감정 표현, 반응 방식을 다르게 한 새로운 한국어 구어체 대화 데이터를 생성하세요.

Important Definition

여기서 "증강"이란 새로운 사건을 창작하는 것이 아닙니다.
반드시 원본 대화와 같은 상황, 같은 목적, 같은 사기 수법, 같은 전개를 유지한 채
표현만 다르게 바꾼 변형 데이터(variation)를 만드는 것입니다.

Input

[Original Data]

<원본 JSON이 여기에 삽입됨>

Preserve Strictly

다음 요소는 반드시 유지하세요.

1. 전체 통화의 목적과 결론
2. 사기 수법의 종류
 - 예: 기관 사칭, 계좌 이체 유도, 앱 설치 유도, 개인정보 요구 등
3. 대화의 핵심 흐름
 - 예: 신분 확인 → 위기 조성 → 압박 → 행동 유도
4. 화자 간 역할 관계
 - 사기 시도 화자와 상대 화자의 역할은 유지
5. 전체 라벨
 - conversation_label은 원본과 동일해야 함

You May Vary

다음 요소는 자연스럽게 변경해도 됩니다.

1. 어휘와 표현
2. 문장 길이와 문장 구조
3. 존댓말/반말 수준의 미세한 차이
4. 피해자 반응의 뉘앙스

- 더 의심스럽게, 더 당황하게, 더 순진하게 등

5. 발화 개수

- 자연스러우면 문장을 합치거나 나눌 수 있음
- 발화 개수는 원본 대비 크게 벗어나지 않도록 하세요.
- 가능하면 원본 발화 수의 70%~130% 범위 내에서 조정하세요.
- 지나치게 짧게 요약하거나, 불필요하게 여러 발화를 추가하지 마세요.

Hard Constraints

다음은 절대 금지입니다.

1. 새로운 사건이나 새로운 사기 수법 추가 금지
2. 원본에 없는 핵심 행동 목표 추가 금지
 - 예: 원본에 계좌이체만 있는데 앱 설치까지 추가 금지
3. 원본에 없는 핵심 정보 삭제 금지
4. `conversation_label` 변경 금지
5. 출력의 `speaker` 값은 반드시 S1, S2만 사용해야 합니다.
 - 원본의 화자 역할 관계는 유지하되, 표기는 독립적인 S1, S2로 정규화하세요.
6. JSON 외의 텍스트 출력 금지
7. 설명, 주석, 마크다운 코드블록 출력 금지
8. 원본 대화에 포함된 핵심 행위 유발 정보(예: 기관 사칭, 계좌 이체 요구, 앱 설치 요구, 개인정보 요청, 인증 요구, 협박성 안내)는 완곡하게 바꾸더라도 의미상 반드시 유지해야 합니다.

Style Requirements

1. 반드시 자연스러운 한국어 구어체로 작성하세요.
2. 실제 통화처럼 짧고 현실적인 발화를 사용하세요.
3. 각 `variation`은 서로 문체가 충분히 다르게 보이도록 작성하세요.
4. 단순한 단어 치환만 하지 말고, 문장 구조와 반응 방식도 바꾸세요.
5. 그러나 의미는 원본과 동일해야 합니다.
6. 지나치게 문어체이거나 요약된 문장은 피하세요.

Quality Requirements

생성 전 아래를 스스로 점검한 뒤 출력하세요.

1. 원본과 동일한 사기 시나리오인가?
2. 핵심 설득 흐름이 유지되는가?

3. 각 variation이 서로 충분히 다른가?
4. 한국어 통화체로 자연스러운가?
5. JSON 형식이 정확한가?
6. idx가 0부터 순차적으로 증가하는가?
7. 모든 utterance에는 idx, speaker, text가 반드시 포함되어야 합니다.
8. text는 빈 문자열이면 안 됩니다.
9. utterances는 최소 2개 이상의 발화를 포함해야 합니다.
10. JSON 필드명은 예시와 정확히 동일해야 합니다.

Output Format

반드시 JSON 리스트만 반환하세요.

리스트 길이는 정확히 3이어야 합니다.

각 원소는 아래 형식을 따르세요.

```
[
  {
    "conversation_id": 1000011,
    "conversation_label": 1,
    "utterances": [
      { "idx": 0, "speaker": "S1", "text": "..." },
      { "idx": 1, "speaker": "S2", "text": "..." }
    ]
  },
  {
    "conversation_id": 1000012,
    "conversation_label": 1,
    "utterances": [
      { "idx": 0, "speaker": "S1", "text": "..." },
      { "idx": 1, "speaker": "S2", "text": "..." }
    ]
  },
  {
    "conversation_id": 1000013,
    "conversation_label": 1,
    "utterances": [
      { "idx": 0, "speaker": "S1", "text": "..." },
      { "idx": 1, "speaker": "S2", "text": "..." }
    ]
  }
]
```

```

    ]
  }
]

# Additional Rules for conversation_id
- 원본 conversation_id를 바탕으로 새로운 정수형 id를 생성하세요.
- 증강 데이터의 conversation_id는 반드시 7자리 정수여야 합니다.
- 원본이 100001이면 증강본은 1000011, 1000012, 1000013 형태를 사용하세요.
- 3개 모두 서로 달라야 합니다.

# Final Instruction
원본 데이터와 같은 보이스피싱 시나리오를 유지한 변형 데이터 3개를 생성하세요.
반드시 JSON 리스트만 반환하세요.

# Diversity Requirement Across 3 Variations
세 개의 variation은 서로 다음 방향으로 차이를 가지도록 하세요.
1. Variation 1: 원본과 가장 가까운 보수적 변형. 핵심 표현만 자연스럽게 바꾼다.
2. Variation 2: 피해자의 반응을 더 의심스럽고 질문이 많은 방향으로 조정한다.
3. Variation 3: 사기 화자의 말투를 더 압박적이거나 더 친절하게 바뀌되, 행동 유도 목적은 동일하게 유지한다.
단, 세 variation 모두 핵심 시나리오와 사기 수법은 동일해야 합니다.

```

4. 문장별 라벨링

- raw, augmented 디렉토리에 존재하는 모든 json 파일들에 대해 아래의 프롬프트와 Gemini 2.5 Flash-Lite 모델을 활용하여 문장별 라벨링을 진행한다.

▼ 프롬프트

[SYSTEM ROLE]

당신은 금융 보안 전문가이자 AI 데이터 라벨러입니다.

당신의 임무는 주어진 전화 통화 내역을 순서대로 분석하여, 각 발화 (utterance)에 보이스피싱 판단에 중요한 범죄 의도가 포함되어 있는지 판정하고, 지정된 JSON 포맷으로 반환하는 것입니다.

[TASK DESCRIPTION]

입력된 통화 대화문의 각 발화에 대해 label을 0 또는 1로 부여하십시오.

- 1 (Suspicious/Phishing):

해당 발화가 현재 시점까지의 문맥에서 보이스피싱을 판단하는 데 중요한 범죄 신호를 포함하는 경우.

- 0 (Normal):

일상적인 대화, 단순 질문, 인사, 일반적인 응답, 또는 현재 시점까지의 문맥에서 보이스피싱 핵심 의도가 드러나지 않는 경우.

[ABSOLUTE TEMPORAL RULE]

각 발화의 라벨은 반드시 해당 발화와 그 이전까지의 문맥만 사용하여 판단하십시오.

즉, idx=i인 발화의 label을 판단할 때 사용할 수 있는 정보는:

- utterances[0], utterances[1], ..., utterances[i]

사용할 수 없는 정보는:

- utterances[i+1], utterances[i+2], ... 이후의 미래 발화

반드시 발화를 idx가 작은 순서부터 순차적으로 읽는다고 가정하고 판단하십시오.

[DECISION CRITERIA]

다음 중 하나라도 해당하면 label=1로 판정하십시오.

1. 금전 요구 및 자금 이동 유도

- 계좌 이체, 송금, 현금 전달, 대출 상환, 상품권 구매, 안전계좌 이체 등을 지시하거나 유도하는 발화

1. 악성 앱 설치 또는 특정 접속 유도

- 원격 제어 앱 설치

- 악성 앱 설치
- 특정 URL 클릭
- 특정 사이트 접속
- 앱 마켓이 아닌 경로의 설치 유도

1. 민감 정보 요구

- 비밀번호, OTP, 보안카드 번호, 주민등록번호 전체, 계좌번호, 인증번호, 카드번호 등 민감 정보를 직접 요구하는 발화

1. 심리적 압박 및 고립 유도

- 검찰, 경찰, 금감원, 은행, 카드사 등 기관 사칭
- 구속, 수사, 사건 연루, 체포, 압수수색 등의 위협
- “지금 당장”, “즉시”, “절대 끊지 마세요”, “비밀로 하세요”, “혼자 계세요” 등의 긴급성/고립성/압박성 발화

1. 이전 문맥과 결합될 때 범죄 의도를 실질적으로 강화하는 발화

- 현재 문장만 보면 짧거나 모호해도, 이전 문맥과 합쳐졌을 때 금전 요구, 정보 탈취, 앱 설치, 압박, 사칭의 실행을 명확히 진전시키면

1

[WHAT SHOULD BE 0]

다음은 원칙적으로 label=0입니다.

1. 일반 인사, 맞장구, 단순 응답

- 예: “네”, “아 네”, “여보세요”, “무슨 일이시죠?”

1. 피해자의 일반 반응

- 놀람, 질문, 확인 요청, 거절, 망설임 등

1. 보이스피싱 통화 내에서도 범죄 핵심 의도가 직접 드러나지 않는 단순 연결 발화

- 예: “잠시만요”, “확인해보겠습니다”, “네 맞습니다”

단, 이전 문맥과 결합했을 때 현재 발화가 실질적 압박/유도 역할을 하면 1이 될 수 있습니다.

1. 입력의 conversation_label이 1이라도, 모든 발화가 자동으로 1이 되는 것은 아님

[CONTEXT AND BIAS GUIDELINES]

1. 현재 발화의 의미를 판단할 때는 현재 발화와 이전까지의 문맥만 참고하십시오.

2. conversation_label은 정답을 미리 알려주는 힌트가 아닙니다. 개별 발화를 독립적으로 판정하십시오.

3. speaker 값은 판단 근거로 사용하지 마십시오.
4. 특정 speaker가 사기범 또는 피해자라고 가정하지 마십시오.
5. 오직 발화 내용과 현재 시점까지의 문맥만으로 판단하십시오.

[IMPORTANT LABELING PRINCIPLE]

label=1은 “이 문장이 보이스피싱 판단에 중요한 범죄 신호를 포함하는가”를 의미합니다.

따라서 다음을 구분해야 합니다.

- 기관 사칭, 긴급 압박, 금전 요구, 정보 요구, 앱 설치 유도처럼 핵심 신호가 있는 문장 → 1
- 같은 보이스피싱 통화 안에 있어도 단순 연결, 일반 응답, 의미 없는 짧은 반응 → 0

[INPUT FORMAT]

입력은 아래 필드를 포함한 단일 JSON 객체입니다.

- conversation_id: 대화 고유 ID (정수 또는 문자열)
- conversation_label: 대화 전체 라벨 (0: 정상, 1: 피싱)
- utterances: 대화 내용이 담긴 객체 리스트
 - idx: 정수
 - speaker: 문자열
 - text: 문자열

[OUTPUT FORMAT]

입력된 JSON 구조를 유지하되, utterances 내부 각 객체에 label 필드를 추가하여 반환하십시오.

반환은 반드시 단일 JSON 객체여야 하며, JSON 외의 설명 문장이나 코드블록을 출력하지 마십시오.

형식은 아래와 같습니다.

```
{
  "conversation_id": <입력값 유지>,
  "conversation_label": <입력값 유지>,
  "utterances": [
    {
      "idx": 0,
```

```

"label": 1,
"speaker": "A",
"text": "...",
},
{
"idx": 1,
"label": 0,
"speaker": "B",
"text": "...",
}
]
}

```

[OUTPUT RULES]

1. conversation_id는 입력값을 그대로 유지하십시오.
2. conversation_label은 입력값을 그대로 유지하십시오.
3. idx는 입력값을 그대로 유지하십시오. 문자열로 바꾸지 마십시오.
4. speaker는 입력값을 그대로 유지하십시오.
5. text는 입력값을 그대로 유지하십시오.
6. 추가 필드는 label만 허용됩니다.
7. label은 반드시 정수 0 또는 1이어야 합니다.
8. JSON 필드명은 정확히 유지하십시오.
9. JSON 외의 텍스트는 절대 출력하지 마십시오.

[FEW-SHOT EXAMPLE 1]

User Input:

```

{
"conversation_id": 1024,
"conversation_label": 1,
"utterances": [
{"idx": 0, "speaker": "A", "text": "서울지검 첨단범죄수사팀  
김민수 수사관입니다."},
{"idx": 1, "speaker": "B", "text": "네 무슨 일이시죠?"},
{"idx": 2, "speaker": "A", "text": "지금 당장 계좌의 잔액을  
안전계좌로 이체하셔야 합니다."}
]
}

```

```
}
```

Model Output:

```
{
  "conversation_id": 1024,
  "conversation_label": 1,
  "utterances": [
    {"idx": 0, "label": 1, "speaker": "A", "text": "서울지검  
첨단범죄수사팀 김민수 수사관입니다."},
    {"idx": 1, "label": 0, "speaker": "B", "text": "네 무슨  
일이시죠?"},
    {"idx": 2, "label": 1, "speaker": "A", "text": "지금 당장  
계좌의 잔액을 안전계좌로 이체하셔야 합니다."}
  ]
}
```

[FEW-SHOT EXAMPLE 2]

User Input:

```
{
  "conversation_id": 2048,
  "conversation_label": 1,
  "utterances": [
    {"idx": 0, "speaker": "A", "text": "검찰청입니다. 본인 명의  
계좌가 범죄에 연루됐습니다."},
    {"idx": 1, "speaker": "B", "text": "네? 무슨 말씀이세요?"},
    {"idx": 2, "speaker": "A", "text": "잠시만요. 기록을 확인해  
보겠습니다."},
    {"idx": 3, "speaker": "A", "text": "절대 전화를 끊지 마시고  
혼자 계셔야 합니다."}
  ]
}
```

Model Output:

```
{
  "conversation_id": 2048,
  "conversation_label": 1,
  "utterances": [
```

```
{ "idx": 0, "label": 1, "speaker": "A", "text": "검찰청입  
니다. 본인 명의 계좌가 범죄에 연루됐습니다." },  
{ "idx": 1, "label": 0, "speaker": "B", "text": "네? 무슨  
말씀이세요?" },  
{ "idx": 2, "label": 0, "speaker": "A", "text": "잠시만요.  
기록을 확인해보겠습니다." },  
{ "idx": 3, "label": 1, "speaker": "A", "text": "절대 전화  
를 끊지 마시고 혼자 계셔야 합니다." }  
]  
}
```

5. 데이터 split

- 아래의 규칙들을 적용하여 원본 데이터를 train/valid/test로 split 한다.
 - split은 원본 통화 json 단위로 수행한다.
 - train : valid : test = 70 : 15 : 15
 - split은 코드에서 무작위로 수행하되, 실험 재현성을 위해 랜덤 시드를 고정한다.
 - 각 split에 배정된 원본 통화에서 생성되는 통화 전체 샘플과 누적 prefix 샘플은 모두 동일한 split에 속한다.
 - 비정상 데이터의 증강은 train에 배정된 raw abnormal 통화에 대해서만 수행하고, valid와 test에서는 성능 평가를 위해 원본 통화만 사용한다.

6. 누적 prefix 샘플

- 목적 및 개요
 - 통화가 아직 끝나지 않았을 때 현재까지 내용을 보고 위험도를 추론하는 법을 학습하기 위한 데이터이다.
 - 즉, 최종적으로 통화 전체를 보고 판정하는 것이 아니라, 통화 진행 중 특정 시점까지의 정보만 보고 현재 위험 신호를 감지하는 능력을 학습하는 데 목적이 있다.
- 생성 방식
 - 하나의 통화 내역 json 파일(라벨링 완료 파일)로부터 여러 개 생성된다.
 - 발화를 앞에서부터 일정 개수(5)씩 누적하는 방식으로 prefix를 구성한다.
 - step의 크기는 5로 설정한다.

- 즉, 0~4, 0~9, 0~14, 0~끝 다음과 같이 생성된다.
- 누적 prefix 샘플의 정답 라벨은 `target_label` 필드를 사용한다.
 - 정상 통화
 - `conversation_label = 0` 인 경우 모든 prefix 샘플의 `target_label`은 0으로 설정한다.
 - 이를 통해 문장별 라벨링 과정에서 일부 노이즈가 존재하더라도, 정상 통화 전체가 잘못 1로 전파되는 것을 방지한다.
 - 비정상 통화
 - 보이스피싱 통화에서 처음 위험 문장(label = 1)이 등장하기 전까지는 `target_label`을 0, 이후로는 1로 설정한다.
- ▼ 누적 prefix 샘플의 json 스키마

```
{
  "sample_id": "prefix_200001_0009",
  "sample_type": "prefix",
  "parent_conversation_id": 200001,
  "conversation_label": 1,
  "target_label": 0,
  "prefix_end_idx": 9,
  "instruction": "현재까지의 통화 내용만 보고 보이스피싱 위험 여부를 판단하라. 위험하면 1, 아니면 0만 출력하라.",
  "input": "...",
  "output": "0"
}
```

- 누적 prefix 샘플의 json을 하나의 jsonl 파일로 병합한다.

7. samples/conversation 파일 전처리

- 누적 prefix 샘플의 json 스키마와 동일하게 전체 통화 내역 샘플의 스키마를 수정한다.
- ▼ json 스키마

```
{
  "sample_id": "conversation_200001",
  "sample_type": "conversation",
```

```
"parent_conversation_id": 200001,  
"conversation_label": 1,  
"target_label": 1,  
"prefix_end_idx": null,  
"instruction": "다음 통화가 보이스피싱인지 정상인지 판단하라.  
보이스피싱이면 1, 정상이면 0만 출력하라.",  
"input": "A: 저희는 은행입니다.\nB: 무슨 일이죠?\nA: 긴급히  
이체하셔야 합니다.",  
"output": "1"  
}
```

- 전체 통화 내역 샘플을 하나의 jsonl 파일로 병합한다.

8. 최종 train / valid / test 데이터 생성

- samples 디렉토리 내부의 conversation과 prefix를 각 split 별로 합쳐 최종적으로 split별로 하나의 jsonl 파일을 생성한다.